

Code as Policies: 面向具身控制的语言模型程序

Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, Andy Zeng

Robotics at Google

摘要——在代码补全任务上训练的大语言模型（LLM）已被证明能够根据文档字符串合成简单的 Python 程序 [1]。我们发现，这些会写代码的 LLM 可以被重新用于根据自然语言指令编写机器人策略代码。具体而言，策略代码可以表达为处理感知输出（例如来自目标检测器 [2]、[3]）并为控制原语 API 设置参数的函数或反馈循环。当以若干示例语言指令（写成注释）及其对应策略代码作为输入（少样本提示）时，LLM 能够接收新指令，并自主地重新组合 API 调用来生成相应的新策略代码。通过串联经典逻辑结构并调用第三方库（如 NumPy、Shapely）进行算术运算，以这种方式使用的 LLM 所编写的机器人策略能够 (i) 表现出空间几何推理能力，

(ii) 泛化到新指令，以及 (iii) 根据上下文为模糊描述（如“更快”）给出精确数值（例如速度），即具备行为常识。本文提出 Code as Policies: 一种以机器人为中心的语言模型生成程序（LMP）表述方式，既能表示反应式策略（如阻抗控制器），也能表示基于航点的策略（基于视觉的抓取放置、基于轨迹的控制），并在多个真实机器人平台上得到验证。我们方法的核心是提示分层代码生成（递归定义尚未定义的函数），它不仅能写出更复杂的代码，还将 HumanEval [1] 基准的最新水平提升至解决 39.8% 的问题。

代码与视频见 <https://code-as-policies.github.io>

一、引言

使用语言的机器人需要将语言接地（或情境化），以指称物理世界，并在词语、感知与动作之间建立联系 [4]。经典方法通过词法分析提取语义表示来为策略提供信息 [5] - [7]，但它们往往难以处理未见过的指令。较新的方法端到端地学习接地（从语言到动作）[8] - [10]，但需要大量训练数据，而在真实机器人上获取这些数据代价高昂。与此同时，自然语言处理的最新进展表明，在互联网规模数据上预训练的大语言模型（LLM）[11] - [13] 具备开箱即用的能力 [14] - [16]，可用于使用语言的机器人，例如无需额外微调即可根据自然语言指令规划一系列步骤 [16] - [18]。这些步骤可以通过一组固定技能（即用行为克隆或强化学习预训练的策略）的价值函数接地到真实机器人的可行性上 [19] - [21]。这一抽象虽有前景，却使 LLM 无法直接影响感知-动作反馈回路，从而难以以下述方式接地语言：

(i) 泛化共享感知与动作的反馈模式，例如从“把苹果放在橙子上”到“看到橙子时就把苹果放下”；(ii) 在控制中表达常识先验，例如“动作快一点”、“推得更用力”；或 (iii) 理解空间关系，如“把苹果往左移一点”。结果是，每加入一项新技能（以及新的接地模式）都需要额外数据并重新训练——数据负担依然存在，只是转移到了技能获取阶段。这促使我们提出问题：

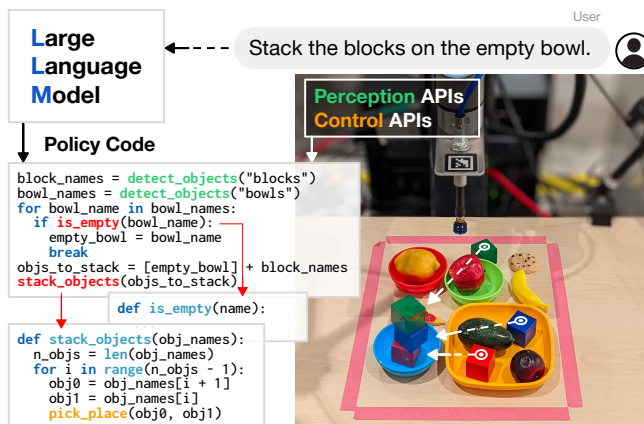


图 1: 给定示例（通过少样本提示），机器人可以利用会写代码的大语言模型（LLM）把自然语言指令翻译成机器人策略代码，这些代码处理感知输出、为控制原语设置参数、递归地为未定义函数生成代码，并泛化到新任务。LLM 能否超越“规划技能序列”而发挥更大作用？本文发现，会写代码的 LLM [1]、[11]、[22] 擅长走得更远：统筹规划、策略逻辑与控制。在代码补全上训练的 LLM 已被证明能从文档字符串合成 Python 程序。我们发现这些模型可以被重新用于根据自然语言指令（写成注释）编写机器人策略代码。策略代码可以表达处理感知输出（如开放词汇目标检测器 [2]、[3]）并为控制原语 API 设置参数的函数或反馈循环（见图 1）。当提供若干示例语言指令及其对应策略代码（少样本提示，灰色）时，LLM 能接收新指令（绿色），并自主重组 API 调用来生成新的策略代码（高亮）：

```
# if you see an orange, move backwards.
if detect_object("orange"):
    robot.set_velocity(x=-0.1, y=0, z=0)
# move rightwards until you see the apple.
while not detect_object("apple"):
    robot.set_velocity(x=0, y=0.1, z=0)
```

写代码的模型可以表达各种算术运算以及由语言接地的反馈循环。它们不仅能泛化到新指令，而且由于在数十亿行代码和注释上训练过，还能依据上下文为模糊描述（“更快”、“往左”）给出精确数值（如速度），从而引出行为常识：

```
# do it again but faster, to the left, and with a banana.
while not detect_object("banana"):
    robot.set_velocity(x=0, y=-0.2, z=0)
```

把代码表示为策略继承了 LLM 的诸多优势：不仅能理解自然语言，还能仅通过把 "say(text)" 作为一个可用的动作原语 API，就参与人机对话与问答：

```
# tell me why you stopped moving.
robot.say("I stopped moving because I saw a banana.")
```

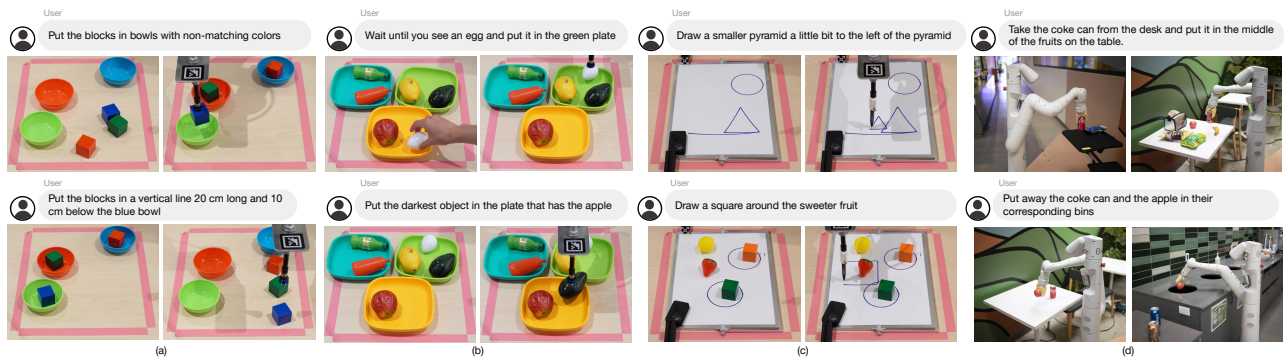


图 2：Code as Policies 能够在多种领域和机器人上遵循自然语言指令：桌面操作 (a)-(b)、二维图形绘制 (c)，以及使用 Everyday Robots 机器人在厨房中进行移动操作 (d)。我们的方法让机器人无需额外训练，仅用现成模型和少样本提示就能进行空间几何推理、解析物体关系并形成多步行为。完整视频与更多任务见 code-as-policies.github.io。我们提出 Code as Policies (CaP)：一种在真实系统上执行的、以机器人中心的语言模型生成程序 (LMP) 表述方式。Python 风格的 LMP 可以用以下手段表达复杂策略：

- 经典逻辑结构，例如顺序、选择 (if/else) 和循环 (for/while)，在运行时组装新行为。
- 第三方库，例如用 NumPy 插值点、用 Shapely 分析与生成形状以进行空间几何推理等。

LMP 可以是分层的：被提示去递归定义新函数，随时间积累自己的函数库，并自我构建一个动态代码库。我们在多个机器人系统上证明，LLM 能自主理解语言指令并生成 LMP，用以表示反应式底层策略（例如 PD 或阻抗控制器）和基于航点的策略（例如基于视觉的抓取放置或基于轨迹的控制）。我们的主要贡献是：(i) Code as Policies：一种利用 LLM 编写机器人代码的表述方式；(ii) 一种分层代码生成方法，在机器人与标准代码生成问题上均提升了最新水平，在 HumanEval [1] 上达到 39.8% P@1；(iii) 一个用于评估未来语言模型在机器人代码生成问题上表现的新基准；以及 (iv) 一系列消融实验，分析 CaP 如何改善泛化指标 [23]，并表明它遵循尺度定律——模型越大表现越好。Code as Policies 为连接词语、感知与动作提供了新途径，可应用于人机交互，但也存在局限，我们将在第五节讨论。完整提示词与生成输出见附录，附录连同更多结果、视频和代码可在 code-as-policies.github.io 获取。

二、相关工作

通过语言控制机器人 由来已久，早期就有通过对自然语言进行词法解析实现人机交互的演示 [5]。语言不仅是非专业人员与机器人交互的界面 [24]、[25]，也是把泛化能力组合式地扩展到新任务的手段 [9]、[17]。相关文献浩繁（综述可参见 Tellex 等 [4] 与 Luketina 等 [26]），但近期工作大致可分为高层解释（如语义解析 [25]、[27] – [32]）、规划 [14]、[17]、[18]，以及底层策略（如基于模型的方法 [33] – [35]、模仿学习 [8]、[9]、[36]、[37]，或强化学习 [38] – [42]）几类。与之相比，我们的工作聚焦于 LLM 的代码生成能力，并把生成的程序作为控制机器人的一种富有表达力的方式。

大语言模型 展现出令人印象深刻的零样本推理能力：从规划 [14] 到编写数学程序 [43]；从解答科学问题 [44] 到用训练好的验证器 [45] 解决数学应用题。这些能力还可以通过 Least-to-Most [46]、Think-Step-by-Step [15] 或 Chain-of-Thought [47] 等提示方法进一步提升。与本文最相关的是那些无需额外训练即把 LLM 能力用于机器人智能体的工作。例如，Huang 等人通过文本补全与语义翻译把自然语言指令分解为可执行动作序列 [14]；SayCan [17] 则通过联合解码 LLM 并以来自价值函数的技能可行性 [20] 加权，为机器人生成可行计划。Inner Monologue [18] 把成功检测器或其他视觉语言模型的输出纳入 LLM 规划，并利用其反馈重新规划。Socratic Models [16] 用视觉语言模型把感知信息（青色）替换进生成计划的语言提示，并使用语言条件策略，例如用于抓取 [36]。下面的例子说明了我们的方法与上述先前工作的定性差异。当任务是“把可乐罐往右移一点”时：

LLM Plan [14], [17], [18]

1. Pick up coke can
2. Move a bit right
3. Place coke can

Socratic Models Plan [16]

```
objects = [coke can]
1. robot.grasp(coke can) open vocab
2. robot.place_a_bit_right()
```

先前工作生成的计划假定存在一个能让机器人“把物体往右移一点”的技能。我们的方法不同之处在于，它用 LLM 直接生成在机器人上运行的策略代码（计划嵌套其中），从而避免了为计划中每一步都预定义策略的要求：

```
Code as Policies (ours)
while not obj_in_gripper("coke can"):
    robot.move_gripper_to("coke can")
robot.close_gripper()
pos = robot.gripper.position
robot.move_gripper(pos.x, pos.y+0.1, pos.z)
robot.open_gripper()
```

Our approach (CaP) not only leverages logic structures to specify feedback loops, but it also parameterizes (and write parts of) low-level control primitives. CaP alleviates the need to collect data and train a fixed set of predefined skills or language-conditioned policies – which are expensive and often remain domain-specific.

Code generation has been explored with LLMs [1], [48] and without [49]. Program synthesis has been demonstrated to be capable of drawing simple figures [50] and generating policies

用于解决二维任务 [51]。我们在这些工作的基础上进一步展示：(i) 会写代码的 LLM 能带来新的推理能力（例如借助对第三方库的熟悉来编码空间关系），而无需先前工作 [35]、[36]、[52] – [56] 所需的额外训练；(ii) 分层代码编写（受递归摘要 [57] 启发）提升了代码生成的最新水平。我们还提出了一个以机器人为主题的新代码生成基准，用于评估未来语言模型在机器人领域的表现。

三、方法

本节刻画预训练 LLM 在多大程度上可以被提示去生成“代码即策略”——表示为一组语言模型程序 (LMP)。广义上，我们用 LMP 一词指任何由语言模型生成并在系统上执行的程序。本文研究 Code as Policies，这是一类把语言指令映射为代码片段的 LMP，这些代码片段 (i) 对感知输入（来自传感器或建立在传感器之上的模块）做出反应，(ii) 为控制原语 API 设置参数，并且

(iii) 直接在机器人上编译执行，例如：

```
# stack the blocks in the empty bowl.
empty_bowl_name = parse_obj('empty bowl')
block_names = parse_obj('blocks')
obj_names = [empty_bowl_name] + block_names
stack_objs_in_order(obj_names=obj_names)
```

输入指令写成注释（绿色），可以由人提供，也可以由另一个 LMP 编写。LLM 的预测输出（高亮）应为有效的 Python 代码，以自回归方式生成 [11]、[12]。LMP 通过少样本示例提示来生成不同的子程序，这些子程序可以处理目标检测结果、构建轨迹或串接控制原语。LMP 可以分层生成：组合已知函数（例如 `get_obj_names()` 使用感知模块），或调用其他 LMP 来定义尚未定义的函数：

```
# define function stack_objs_in_order(obj_names).
def stack_objs_in_order(obj_names):
    for i in range(len(obj_names) - 1):
        put_first_on_second(obj_names[i + 1], obj_names[i])
```

其中 `put_first_on_second` 是一个已有的开放词汇抓取放置原语（例如 CLIPort [36]）。对于新的本体，这些主动函数调用可以替换为表示智能体动作空间的可用控制 API（例如 `set_velocity`）。带有详尽变量名的分层代码生成可以看作通过函数式编程实现的一种思维链提示变体 [47]。由 LMP 定义的函数可以随时间逐步积累，新的 LMP 可以引用先前构建的函数来扩展策略逻辑。执行 LMP 时，我们首先确保代码可以安全运行：没有 `import` 语句、没有以 `_` 开头的特殊变量，也没有对 `exec` 和 `eval` 的调用。然后以代码作为输入字符串调用 Python 的 `exec` 函数，并传入构成该代码执行作用域的两个字典：(i) `globals`，包含生成代码可能调用的所有 API；(ii) `locals`，一个空字典，将在 `exec` 过程中被定义的变量和新函数填充。如果期望 LMP 返回一个值，我们在 `exec` 结束后从 `locals` 中获取。

A. 提示语言模型程序

生成 LMP 的提示词包含两个要素：1. 提示 (Hints)，例如告知 LLM 有哪些 API 可用的 `import` 语句，以及说明如何使用这些 API 的类型提示。

```
import numpy as np
from utils import get_obj_names, put_first_on_second
```

2. 示例 (Examples) 即指令到代码的配对，以少样本“演示”展示自然语言指令应如何转换为代码。这些示例可以包括算术运算、调用其他 API 以及编程语言的其他特性。指令写成注释，直接置于对应解答代码块之前。我们可以通过把新的指令与响应逐步追加到提示词中来维持一个 LMP“会话”，使后续指令能够回指先前指令，例如“撤销上一个动作”。

B. 语言模型程序示例（底层）

理解 LMP 最好的方式或许是看例子，下面这一节从简单的纯 Python 指令逐步过渡到能完成机器人任务的更复杂指令。除特别说明外，本文所有示例与实验均使用 OpenAI Codex code-davinci-002，温度为 0（即确定性贪心解码）。此处提示词（灰色）以一条 Hint 开头，表明我们在写 Python；随后给出一个 Example 以指定返回值的格式，返回值将赋给名为 `ret_val` 的变量。输入指令为绿色，生成输出为高亮：

```
# Python script
# get the variable a.
ret_val = a
# find the sum of variables a and b.
ret_val = a + b
# see if any number is divisible by 3 in a list called xs.
ret_val = any(x % 3 == 0 for x in xs)
```

第三方库。 会写 Python 代码的 LLM 存储着许多流行库的知识。可以提示 LMP 使用这些库来执行复杂指令，而无需把所有代码都写出来，例如用 NumPy 进行基于坐标的空间推理。此处 Hints 包含 `import` 语句，Examples 定义了基本方位。变量名同样重要，用于表明 `pts_np` 和 `pt_np` 是 NumPy 数组。对二维向量的运算意味着这些点也是二维的。示例：

```
import numpy as np
# move all points in pts_np toward the right.
ret_val = pts_np + [0.3, 0]
# move a pt_np toward the top.
ret_val = pt_np + [0, 0.3]
# get the left most point in pts_np.
ret_val = pts_np[np.argmin(pts_np[:, 0]), :]
# get the center of pts_np.
ret_val = np.mean(pts_np, axis=0)
# the closest point in pts_np to pt_np.
ret_val = pts_np[np.argmin(np.sum((pts_np - pt_np)**2, axis=1))]
```

第一方库。 如果函数名有意义并在 Hints/Examples 中给出，LMP 也可以使用训练数据中不存在的第一方库（感知或控制原语 API）。例如（完整提示词见 B.2）：

```

from utils import get_pos, put_first_on_second
...
# move the purple bowl toward the left.
target_pos = get_pos('purple bowl') + [-0.3, 0]
put_first_on_second('purple bowl', target_pos)
objs = ['blue bowl', 'red block', 'red bowl', 'blue block']
# move the red block a bit to the right.
target_pos = get_pos('red block') + [0.1, 0]
put_first_on_second('red block', target_pos)
# put the blue block on the bowl with the same color.
put_first_on_second('blue block', 'blue bowl')

```

Hints 为一个机器人领域导入了两个函数：一个按名称获取物体的二维位置（使用开放词汇目标检测器 [2]），另一个把第一个物体放到第二个目标上，目标可以是物体名称或二维位置。请注意 LMP 适应新指令的能力——第一条指令用“一点”修改了移动幅度，第二条则把物体与“相同颜色”关联起来。语言推理也可以用会写代码的 LLM 进行少样本提示（完整提示词见 B.1），例如把物体名称与自然语言描述（“海洋色的积木”）、类别（“碗”）或历史上下文（“另一块积木”）关联起来：

```

objs = ['blue bowl', 'red bowl', 'red bowl', 'blue block']
# the bowls.
ret_val = ['blue bowl', 'red bowl']
# sea-colored block.
ret_val = 'blue block'
# the other block.
ret_val = 'red block'

```

C. 语言模型程序示例（高层）

控制流。编程语言允许使用 if-else 和循环语句等控制结构。前面我们展示了 LMP 能以列表推导的形式表达 for 循环。这里我们展示它们如何写出 while 循环，从而构成一个简单的反馈策略。注意提示词（与 B.2 相同）中并不包含此类 Examples：

```

# while the red block is to the left of the blue bowl, move it to the
right 5cm at a time.
while get_pos('red block')[0] < get_pos('blue bowl')[0]:
    target_pos = get_pos('red block') + [0.05, 0]
    put_first_on_second('red block', target_pos)

```

LMP 可以 通过嵌套函数调用进行组合。这使得每个提示词可以容纳更多少样本示例以提升功能准确性与覆盖范围，同时不超过 LLM 的最大输入 token 长度。下面的例子（完整提示词见 B.4）生成的响应使用了 parse_obj，这是另一个把物体名称与语言描述关联起来的 LMP：

```

objs = ['red block', 'blue bowl', 'blue block', 'red bowl']
# while the left most block is the red block, move it toward the right.
block_name = parse_obj('the left most block')
while block_name == 'red block':
    target_pos = get_pos(block_name) + [0.3, 0]
    put_first_on_second(block_name, target_pos)
    block_name = parse_obj('the left most block')

```

The `parse_obj` LMP (full prompt in Appendix B.5):

```

objs = ['red block', 'blue bowl', 'blue block', 'red bowl']
# the left most block.
block_names = ['red block', 'blue block']
block_positions = np.array([get_pos(name) for name in block_names])
left_block_name = block_names[np.argmin(block_positions[:, 0])]
ret_val = left_block_name

```

LMP 可以分层地生成函数以供将来复用：

```

import numpy as np
from utils import get_obj_bbox_xyxy
# define function: total = get_total(xs).
def get_total(xs):
    return np.sum(xs)
# define function: get_objs_bigger_than_area_th(obj_names, bbox_area_th).
def get_objs_bigger_than_area_th(obj_names, bbox_area_th):
    return [name for name in obj_names
            if get_obj_bbox_area(name) > bbox_area_th]

```

函数生成可以这样实现：解析 LMP 生成的代码，找出尚未定义的函数，然后调用另一个专门负责函数生成的 LMP 来创建它们。这样一来，提示词和 LMP 生成的代码都可以调用尚未定义的函数。提示词工程师不再需要在 Examples 中给出全部实现细节——代码逻辑的一个“粗略草图”可能就足够了。高层 LMP 还可以遵循良好的抽象实践，避免把所有代码逻辑“摊平”到同一层。这不仅让生成的代码更易读，也如第 IV-A 节所示提升了代码生成性能。在生成函数的函数体内部同样会定位尚未定义的函数。注意在上面的例子中，`get_obj_bbox_area` 并不是给定的 API 调用，而是可以按需生成：

```

# define function: get_obj_bbox_area(obj_name).
def get_obj_bbox_area(obj_name):
    x1, y1, x2, y2 = get_obj_bbox_xyxy(obj_name)
    return (x2 - x1) * (y2 - y1)

```

注意提示词并未明确说明 `get_obj_bbox_xyxy` 返回什么，但函数名暗示它包含一个轴对齐包围盒的最小和最大 xy 坐标，LLM 能够推断出这一点并生成正确代码。在 Python 中，我们通过解析代码块的抽象语法树并检查给定作用域中不存在的函数来实现分层函数生成。我们用负责函数生成的 LMP 编写这些未定义函数并加入作用域。这一过程在生成的函数体上重复进行，以深度优先的方式分层创建新函数。将控制流、LMP 组合与分层

函数生成。下面的例子展示 LMP 如何组合这些能力来遵循更复杂的指令并完成桌面操作领域的任务。为简洁起见省略了提示词，但它们与前面的类似。高层 LMP 生成高层策略行为，并依靠 `parse_obj` 根据语言描述获取物体名称：

```

objs = ['red block', 'blue bowl', 'blue block', 'red bowl']
# while there are blocks with area bigger than 0.2 that are left of the
red bowl, move them toward the right.
block_names = parse_obj('blocks with area bigger than 0.2 that are
left of the red bowl')
while len(block_names) > 0:
    for block_name in block_names:
        target_pos = get_pos(block_name) + np.array([0.1, 0])
        put_first_on_second(block_name, target_pos)
    block_names = parse_obj('blocks with area bigger than 0.2 that are
left of the red bowl')

```

然后，`parse_obj` 使用 `get_objs_bigger_than_area_th`（尚未定义的函数）来完成查询。该函数通过 `parse_obj` 提示词的 Hints 中的 `import` 语句给出，但并未实现。随后，分层函数生成会像上文演示的那样创建这个函数。

```

objs = ['red block', 'blue bowl', 'blue block', 'red bowl']
# blocks with area bigger than 0.2 that are left of the red bowl.
block_names = ['red block', 'blue block']
red_bowl_pos = get_pos('red bowl')
use_block_names = [name for name in block_names
                    if get_pos(name)[0] < red_bowl_pos[0]]
use_block_names = get_objs_bigger_than_area_th(use_block_names, 0.2)
ret_val = use_block_names

```

We describe more on prompt engineering in the Appendix A.

D. 作为策略的语言模型程序

在机器人策略的语境下，LMP 可以根据自然语言指令组合出从感知到控制的反馈逻辑：感知模型的高层输出（状态）可以被程序化地处理，并用来确定底层控制 API 的参数（动作）。关于可用感知和控制 API 的先验信息可以通过 Examples 与 Hints 来引导。这些 API 把 LMP“接地”到真实的机器人系统，感知与控制算法的改进可以直接转化为基于 LMP 的策略能力的提升。例如，在下文的真实世界实验中，我们直接使用最近提出的开放词汇目标检测模型 ViLD [3] 和 MDETR [2] 来获取物体位置和包围盒。基于 LMP 的策略有三个方面好处：（i）能根据未见过的自然语言指令所指定的新任务和新行为调整策略代码与参数；（ii）能借助开放词汇感知系统和/或显著性模型泛化到新物体和新环境；（iii）不需要任何额外的数据采集或模型训练。生成的计划和策略以代码表示，因而可解释，便于修改与复用。把 LMP 用于高层用户交互还继承了 LLM 的优势，包括以常识知识解析富有表达力的自然语言、考虑先前上下文、多语言能力以及参与对话。在接下来的实验部分，我们展示 LMP 在不同机器人和不同任务上的多种实例，呈现该方法灵活的能力与易用性。

四、实验

我们的实验有三个目标：（i）评估使用分层代码生成（在不同语言模型上）的影响并分析泛化模式；（ii）在仿真的语言指令操作任务中把 Code as Policies (CaP) 与基线进行比较；（iii）在不同机器人系统上演示 CaP，展示其灵活性与易用性。更多实验见附录，例如生成用于平衡倒立摆和执行末端阻抗控制的反应式控制器（附录 F）。附录还包含所有实验的提示词与响应。复现这些实验的视频和 Colab Notebook 可在网站上找到。由于开放式任务难以评估且缺乏可比基线，对使用 CaP 的机器人系统的定量评估仅限于 IV-D 中一组受限的仿真任务；而在 IV-B、IV-C 和 IV-E 中，我们演示系统支持的全部指令范围，不做定量评估。

表 I: 分层代码生成在 RoboCodeGen 上解决了更多问题（通过率 %），并随尺度定律（模型参数量增加）而提升。

Method	GPT-3 [12]		Codex [1]	
	6.7B	175B	cushman	davinci
Flat	3	68	54	81
Hierarchical	5	84	57	95

表 II: 分层代码生成在 HumanEval [1] 的通用编程问题上表现更好（通过率 %）。Greedy 指以 temperature=0. 解码 LLM；P@N 以 temperature=0.8. 评估 N 个样本中的正确性。

	Greedy	P@1	P@10	P@100
Flat	45.7	34.9	75.1	90.9
Hierarchical	53.0	39.8	80.6	95.7

A. 代码生成基准上的分层 LMP

我们在两个代码生成基准上评估我们的代码生成方法：（i）以机器人为主题的 RoboCodeGen，以及

（ii）由标准代码生成问题组成的 HumanEval [1]。

RoboCodeGen: 我们提出一个包含 37 个函数生成问题的新基准，它与以往代码生成基准有几个关键区别：（i）以机器人为主题，问题涉及空间推理（例如找到离一组点最近的点）、几何推理（例如判断一个包围盒是否包含在另一个之中）和控制（例如 PD 控制）；（ii）允许并鼓励使用第三方库（如 NumPy）；（iii）给出的函数头没有文档字符串也没有显式类型提示，因此 LLM 需要推断并使用常见约定；（iv）允许使用尚未定义的函数，这些函数可通过分层代码生成创建。基准问题示例见附录 E。我们在 OpenAI API 可访问的四个 LLM 上进行评估¹。与标准基准 [1] 一样，评估指标是生成代码通过人工编写单元测试的百分比。见表 I。领域专用语言模型（Codex 模型）总体表现更好。在同一模型家族内，模型越大性能越好。分层方法全面领先，表明让 LLM 把复杂函数分解为层次化部分并分别生成代码是有益的。我们还分析了代码生成性能在 [23] 提出的五种泛化类型上的差异。分层方法对 Productivity（生产性）帮助最大，即新指令需要比 Examples 更长的代码或更多逻辑层次的情形。不过这些提升只出现在两个 davinci 模型上，cushman 上没有，说明需要先达到一定的代码生成能力，分层代码生成才能带来进一步提升。更多结果见附录 E.2。在 HumanEval [1] 上的评估表明，分层代码生成不仅改善策略代码，也改善通用代码。见表 II。所取得的数字高于近期工作 [1]、[11]、[58]。更多细节见附录 D。

B. CaP: 通过生成航点绘制图形

在该领域中，我们让一台真实的 UR5e 机械臂通过生成并跟随一系列二维航点来绘制各种图形。在

¹ 两个文本模型：6.7B 的 GPT-3 模型 [12] 和 175B 的 InstructGPT [22]。两个 Codex 模型 [1]:ushman 与 davinci，均为生成代码而训练。davinci 更大也更好。Codex 模型的规模未公开。

表 III: 各任务族的成功率, 每个任务 50 次试验。

Train/Test	Task Family	CLIPort [36]	NL Planner	CaP (ours)
SA SI	Long-Horizon	78.80	86.40	97.20
SA SI	Spatial-Geometric	97.33	N/A	89.30
UA SI	Long-Horizon	36.80	88.00	97.60
UA SI	Spatial-Geometric	0.00	N/A	73.33
UA UI	Long-Horizon	0.00	64.00	80.00
UA UI	Spatial-Geometric	0.01	N/A	62.00

感知方面, LMP 被赋予检测物体位置的 API, 由现成的开放词汇目标检测器 MDETR [2] 实现。动作方面, 提供一个末端执行器轨迹跟随 API。共有四个 LMP: (i) 解析用户指令、维护会话并调用动作 API; (ii) 从语言描述中解析物体名称; (iii) 从语言描述中解析航点; (iv) 生成新函数。机器人成功执行未见过的语言指令的示例见图 2c。该系统能够引出空间推理, 根据语言指令绘制全新的图形。更多示例展示了解析精确尺寸、修改先前图形以及多步指令的能力, 完整提示词见附录 H。

C. CaP: 桌面操作的抓取放置策略

桌面操作领域让一台 UR5e 机械臂在桌上抓取和放置各种塑料玩具物体。机械臂配备吸盘夹爪和手持 Intel Realsense D435 相机。我们通过 MDETR [2] 提供检测物体存在、位置和包围盒的感知 API, 并提供一个抓取物体并放到目标位置的脚本化原语。提示词与上一领域类似, 只是把轨迹解析换成了位置解析。机器人执行未见过的语言指令的示例见图 2 的 a、b 面板, 展示了对物体描述和空间关系进行推理的能力。其他使用历史上下文 (如“撤销刚才那步”)、通过几何 (如“最小的”) 和空间 (如“最右边的”) 描述来推理物体的指令见附录 I。

D. CaP: 桌面操作仿真评估

我们在来自 [16]、[18] 的仿真桌面操作环境中评估 CaP。该设置让 UR5e 机械臂与 Robotiq 2F85 夹爪操作 10 个彩色积木和 10 个彩色碗。我们沿用了全部 8 个因多步性质而被称为“长程”的任务 (例如“把积木放进颜色匹配的碗里”), 并定义了 6 个需要更具挑战性、更精确的空间几何推理能力的新任务 (例如“把积木摆成一条对角线”)。每个任务由一些属性参数化 (例如“拿起 <obj> 放到 <corner>”), 每次试验时采样。我们把任务指令 (I) 和属性 (A) 划分为“已见” (SI、SA) 和“未见” (UI、UA) 两类, “已见”表示允许出现在提示词中或被训练 (对于有监督基线而言)。更多细节见附录 K。我们考虑两个基线:

(i) 在 3 万条演示上通过模仿学习训练的语言条件多任务 CLIPort [36] 策略, 以及 (ii) 使用自然语言而非代码的少样本提示 LLM 规划器。

结果见表 III。在属性和指令均已见的任务上, CaP 尽管只对每个任务少样本提示了一条示例 rollout, 仍与有监督的 CLIPort 基线表现相当。当任务属性未见时, CLIPort 的性能显著下降, 而基于 LLM 的方法保持相近的表现。在未见任务和属性上, CLIPort 这类端到端系统难以泛化, CaP 则优于直接用语言推理的 LLM ([20] 中也观察到这一点)。此外, 自然语言规划器 [14]、[16]–[18] 并不适用于需要精确数值空间几何推理的任务。我们在附录 C 中进一步展示了用代码推理相对于用自然语言推理 (包括直接问答和思维链 [47]) 的好处, 特别是前者执行精确数值计算的能力。

E. CaP: 移动机器人导航与操作

在该领域中, 一台带有移动底盘和 7 自由度机械臂的机器人需要在真实厨房中执行导航与操作任务。感知方面, LMP 被赋予由 ViLD [3] 实现的目标检测 API。动作方面, 机器人拥有通过名称和坐标导航到位置以及抓取物体的 API。机器人执行未见过的语言指令的示例见图 2。该领域表明 CaP 可以部署在具有不同 API 的不同机器人系统上的现实任务中。它还展示了借助控制结构遵循长程反应式指令以及进行精确空间推理的能力, 这些是先前工作 [16]、[17]、[36] 难以轻易实现的。提示词与更多示例见附录 J。

五、讨论与局限

CaP 在机器人技术栈的一个特定层面上实现泛化: 理解自然语言指令、处理感知输出, 然后为控制原语设置低维输入参数。它适用于感知与控制分解的系统, 并赋予一定程度的泛化能力 (来自预训练 LLM), 而无需端到端学习所需的大规模数据采集。我们的方法还继承了 LLM 与代码编写无关的能力, 例如支持非英语语言或表情符号的指令 (附录 L)。CaP 还能表达跨本体的计划, 根据可用 API 的不同以不同方式完成同一任务 (附录 M)。然而, 这一能力在现有 LLM 上还很脆弱, 可能需要在领域专用代码上训练的更大模型。目前 CaP 受限于: (i) 感知 API 所能描述的范围 (例如迄今没有视觉语言模型能描述一条轨迹是否“颠簸”或“更像 C 形”);

(ii) 可用的控制原语。只有少数命名的原语参数可以在不使提示词过饱和的情况下调整。CaP 也难以理解明显更长、更复杂或与给定 Examples 处于不同抽象层级的指令。在桌面领域, LMP 很难“用积木盖一座房子”, 因为没有关于搭建复杂三维结构的 Examples。我们的方法还假定所有给定指令都是可行的, 并且无法预先判断响应是否正确。

致谢

特别感谢 Vikas Sindhwani、Vincent Vanhoucke 对写作的宝贵反馈, 以及 Chad Boodoo 在运维与硬件方面的支持。

参考文献

- [1] M. Chen, J. Twarek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv:2107.03374*, 2021.
- [2] A. Kamath, M. Singh, Y. LeCun, G. Synnaeve, I. Misra, and N. Carion, "Mdetr-modulated detection for end-to-end multi-modal understanding," in *ICCV*, 2021.
- [3] X. Gu, T.-Y. Lin, W. Kuo, and Y. Cui, "Open-vocabulary object detection via vision and language knowledge distillation," *arXiv:2104.13921*, 2021.
- [4] S. Tellex, N. Gopalan, H. Kress-Gazit, and C. Matuszek, "Robots that use language," *Review of Control, Robotics, and Autonomous Systems*, 2020.
- [5] T. Winograd, "Procedures as a representation for data in a computer program for understanding natural language," *MIT PROJECT MAC*, 1971.
- [6] J. Dzifcak, M. Scheutz, C. Baral, and P. Schermerhorn, "What to do and how to do it: Translating natural language directives into temporal and dynamic logic representation for goal management and action execution," in *ICRA*, 2009.
- [7] Y. Artzi and L. Zettlemoyer, "Weakly supervised learning of semantic parsers for mapping instructions to actions," *TACL*, 2013.
- [8] C. Lynch and P. Sermanet, "Language conditioned imitation learning over unstructured data," *arXiv:2005.07648*, 2020.
- [9] E. Jang, A. Irpan, M. Khansari, D. Kappler, F. Ebert, C. Lynch, S. Levine, and C. Finn, "Bc-z: Zero-shot task generalization with robotic imitation learning," in *CoRL*, 2022.
- [10] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, "Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks," *RA-L*, 2022.
- [11] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, "Palm: Scaling language modeling with pathways," *arXiv:2204.02311*, 2022.
- [12] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *NeurIPS*, 2020.
- [13] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, "Opt: Open pre-trained transformer language models," *arXiv:2205.01068*, 2022.
- [14] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," *arXiv:2201.07207*, 2022.
- [15] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *arXiv:2205.11916*, 2022.
- [16] A. Zeng, A. Wong, S. Welker, K. Choromanski, F. Tombari, A. Purohit, M. Ryo, V. Sindhwani, J. Lee, V. Vanhoucke *et al.*, "Socratic models: Composing zero-shot multimodal reasoning with language," *arXiv:2204.00598*, 2022.
- [17] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," *arXiv:2204.01691*, 2022.
- [18] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," in *arXiv:2207.05608*, 2022.
- [19] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, "Implicit behavioral cloning," in *CoRL*, 2022.
- [20] A. Zeng, "Learning visual affordances for robotic manipulation," Ph.D. dissertation, Princeton University, 2019.
- [21] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke *et al.*, "Scalable deep reinforcement learning for vision-based robotic manipulation," in *CoRL*, 2018.
- [22] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *arXiv:2203.02155*, 2022.
- [23] D. Hupkes, V. Dankers, M. Mul, and E. Bruni, "Compositionality decomposed: How do neural networks generalise?" *JAIR*, 2020.
- [24] C. Breazeal, K. Dautenhahn, and T. Kanda, "Social robotics," *Springer handbook of robotics*, 2016.
- [25] T. Kollar, S. Tellex, D. Roy, and N. Roy, "Toward understanding natural language directions," in *HRI*, 2010.
- [26] J. Luketina, N. Nardelli, G. Farquhar, J. N. Foerster, J. Andreas, E. Grefenstette, S. Whiteson, and T. Rocktäschel, "A survey of reinforcement learning informed by natural language," in *IJCAI*, 2019.
- [27] M. MacMahon, B. Stankiewicz, and B. Kuipers, "Walk the talk: Connecting language, knowledge, and action in route instructions," *AAAI*, 2006.
- [28] J. Thomason, S. Zhang, R. J. Mooney, and P. Stone, "Learning to interpret natural language commands through human-robot dialog," in *IJCAI*, 2015.
- [29] S. Tellex, T. Kollar, S. Dickerson, M. Walter, A. Banerjee, S. Teller, and N. Roy, "Understanding natural language commands for robotic navigation and mobile manipulation," in *AAAI*, 2011.
- [30] D. Shah, B. Osinski, B. Ichter, and S. Levine, "Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action," *arXiv:2207.04429*, 2022.
- [31] C. Matuszek, E. Herbst, L. Zettlemoyer, and D. Fox, "Learning to parse natural language commands to a robot control system," in *Experimental robotics*, 2013.
- [32] J. Thomason, A. Padmakumar, J. Sinapov, N. Walker, Y. Jiang, H. Yedidsion, J. Hart, P. Stone, and R. Mooney, "Jointly improving parsing and perception for natural language commands through human-robot dialog," *JAIR*, 2020.
- [33] S. Nair, E. Mitchell, K. Chen, S. Savarese, C. Finn *et al.*, "Learning language-conditioned robot behavior from offline data and crowd-sourced annotation," in *CoRL*, 2022.
- [34] J. Andreas, D. Klein, and S. Levine, "Learning with latent language," *arXiv:1711.00482*, 2017.
- [35] P. Sharma, B. Sundaralingam, V. Blukis, C. Paxton, T. Hermans, A. Torralba, J. Andreas, and D. Fox, "Correcting robot plans with natural language feedback," *arXiv:2204.05186*, 2022.
- [36] M. Shridhar, L. Manuelli, and D. Fox, "Cliport: What and where pathways for robotic manipulation," in *CoRL*, 2021.
- [37] S. Stepputtis, J. Campbell, M. Phielipp, S. Lee, C. Baral, and H. Ben Amor, "Language-conditioned imitation learning for robot manipulation tasks," *NeurIPS*, 2020.
- [38] Y. Jiang, S. S. Gu, K. P. Murphy, and C. Finn, "Language as an abstraction for hierarchical deep reinforcement learning," *NeurIPS*, 2019.
- [39] P. Goyal, S. Niekum, and R. J. Mooney, "Pixl2r: Guiding reinforcement learning using natural language by mapping pixels to rewards," *arXiv:2007.15543*, 2020.
- [40] G. Cideron, M. Seurin, F. Strub, and O. Pietquin, "Self-educated language agent with hindsight experience replay for instruction following," *DeepMind*, 2019.
- [41] D. Misra, J. Langford, and Y. Artzi, "Mapping instructions and visual observations to actions with reinforcement learning," *arXiv:1704.08795*, 2017.
- [42] A. Akakia, C. Colas, P.-Y. Oudeyer, M. Chetouani, and O. Sigaud, "Grounding language to autonomously-acquired skills via goal generation," *arXiv:2006.07185*, 2020.
- [43] I. Drori, S. Zhang, R. Shuttlesworth, L. Tang, A. Lu, E. Ke, K. Liu, L. Chen, S. Tran, N. Cheng *et al.*, "A neural network solves, explains, and generates university math problems by program synthesis and few-shot learning at human level," *PNAS*, 2022.
- [44] A. Lewkowycz, A. Andreassen, D. Dohan, E. Dyer, H. Michalewski, V. Ramasesh, A. Slone, C. Anil, I. Schlag, T. Gutman-Solo *et al.*, "Solving quantitative reasoning problems with language models," *arXiv:2206.14858*, 2022.
- [45] K. Cobbe, V. Kosaraju, M. Bavarian, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv:2110.14168*, 2021.
- [46] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, O. Bousquet, Q. Le, and E. Chi, "Least-to-most prompting enables complex reasoning in large language models," *arXiv:2205.10625*, 2022.
- [47] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain of thought prompting elicits reasoning in large language models," *arXiv:2201.11903*, 2022.
- [48] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv:2108.07732*, 2021.
- [49] K. Ellis, C. Wong, M. Nye, M. Sable-Meyer, L. Cary, L. Morales, L. Hewitt, A. Solar-Lezama, and J. B. Tenenbaum, "Dreamcoder: Growing generalizable, interpretable knowledge with wake-sleep bayesian program learning," *arXiv:2006.08381*, 2020.
- [50] L. Tian, K. Ellis, M. Kryven, and J. Tenenbaum, "Learning abstract structure for drawing by efficient motor program induction," *NeurIPS*, 2020.
- [51] D. Trivedi, J. Zhang, S.-H. Sun, and J. J. Lim, "Learning to synthesize programs as interpretable and generalizable policies," *NeurIPS*, 2021.
- [52] O. Mees and W. Burgard, "Composing pick-and-place tasks by grounding language," in *ISER*, 2020.
- [53] W. Liu, C. Paxton, T. Hermans, and D. Fox, "Structformer: Learning spatial structure for language-guided semantic rearrangement of novel objects," in *ICRA*, 2022.
- [54] W. Yuan, C. Paxton, K. Desingh, and D. Fox, "Sornet: Spatial object-centric representations for sequential manipulation," in *CoRL*, 2022.
- [55] A. Buckler, L. Figueredo, S. Haddadin, A. Kapoor, S. Ma, and R. Bonatti, "Reshaping robot trajectories using natural language commands: A study of multi-modal data alignment using transformers," *arXiv:2203.13411*, 2022.

- [56] A. Bobu, C. Paxton, W. Yang, B. Sundaralingam, Y.-W. Chao, M. Cakmak, and D. Fox, "Learning perceptual concepts by bootstrapping from human queries," *RA-L*, 2022.
- [57] J. Wu, L. Ouyang, D. M. Ziegler, N. Stiennon, R. Lowe, J. Leike, and P. Christiano, "Recursively summarizing books with human feedback," *arXiv:2109.10862*, 2021.
- [58] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *MAPS*, 2022.
- [59] K. Zakka, A. Zeng, P. Florence, J. Tompson, J. Bohg, and D. Dwibedi, "Xirl: Cross-embodiment inverse reinforcement learning," in *CoRL*. PMLR, 2022.
- [60] A. Ganapathi, P. Florence, J. Varley, K. Burns, K. Goldberg, and A. Zeng, "Implicit kinematic policies: Unifying joint and cartesian action spaces in end-to-end robot learning," *arXiv:2203.01983*, 2022.

APPENDIX

A. Prompt Engineering

Using LMPs to reliably complete tasks via code generation requires careful prompt engineering. While these prompts do not have to be long, they do need to be relevant and specific. Here, we discuss a few general guidelines that we followed while developing prompts for this paper. It is very important for prompts to contain code that has no bugs. Bugs in the prompt lead to unreliable and incorrect responses. Conversely, if the LMP is writing incorrect code for a given Instruction, the prompt engineer should first verify that the prompt, especially the Examples most closely related to the Instruction, is bug-free. To reduce bugs related to syntax errors, one simple method is writing prompts in a code editor with syntax highlighting. There are many cases where the prompt contains variables or functions whose names are ambiguous. To produce reliable responses under these conditions, Examples in the prompt should treat these ambiguities consistently. If a variable named `point` is treated as a `numpy.ndarray` object in one Example and as a `shapely.geometry.Point` object in another, the LMP will not be able to “decide” on which convention to use, resulting in unreliable responses. Another way to handle ambiguity is by providing informal type hints, such as appending `_np` to variable names to indicate its type, or appending it to function names to indicate the type of the returned variable. In general, more specific variable and function names give more consistent results. For using third party libraries, including import statements in the prompt may not be necessary, as we found that LMPs can generate code that calls NumPy and SciPy without them. However, explicit import statements do improve reliability and increase the chance of LMPs using those libraries when the need arises. For using first party libraries, meaningful function names that follow popular conventions (e.g., begin with `set_` and `get_`) and specify return object formats (e.g., `get_bbox_xyxy`) induce more accurate usages. Import statements in the Hints should be formatted as if we were importing functions. For example, in Python this means using `from utils import function_name` instead of `import function_name`. If the latter is used, the LMP may treat the imported name as a package, and the generated code might write `function_name.function_name()`. One type of LMP failures are related to code generation correctness. For example, minor coding mistakes when calling internal or external APIs, such as missing arguments, can be fixed with an Hint or Example demonstrating the correct usage. Incorrect assumptions on variable types can also be fixed in similar fashions. Other coding failures may be addressed by descriptive function names to encourage appropriate library usage (`perform_function_with_np()`) or succinct code logic (`# implement in one line.`) While it is possible to use LLMs to edit code and fix bugs (e.g., by using OpenAI’s code edit API), in our experience this yielded inconsistent results (not always able to correct mistakes, and sometimes changed what the function was doing), so we did not employ this method in our experiments.

B. Method Section Prompts

1) Language-based reasoning: Full prompt:

```
objs = ['green block', 'green bowl', 'yellow block', 'yellow bowl']
# the yellow block.
ret_val = 'yellow block'
# the blocks.
ret_val = ['green block', 'yellow block']
```

2) *First-party*: Full prompt:

```
from utils import get_pos, put_first_on_second
objs = ['gray block', 'gray bowl']
# put the gray block on the gray bowl.
put_first_on_second('gray block', 'gray bowl')
objs = ['purple block', 'purple bowl']
# move the purple bowl toward the left.
target_pos = get_pos('purple bowl') + [-0.3, 0]
put_first_on_second('purple bowl', target_pos)
```

3) Combining language reasoning, third-party, and first-party libraries.: Full prompt:

```
import numpy as np
from utils import get_pos, put_first_on_second
objs = ['cyan block', 'cyan bowl', 'pink bowl']
# put the cyan block in cyan bowl.
put_first_on_second('cyan block', 'cyan bowl')
objs = ['gray block', 'silver block', 'gray bowl']
# place the top most block on the gray bowl.
names = ['gray block', 'silver block']
positions = np.array([get_pos(name) for name in names])
name = names[np.argmax(positions[:,1])]
put_first_on_second(name, 'gray bowl')
objs = ['purple block', 'purple bowl']
# put the purple bowl to the left of the purple block.
target_pos = get_pos('purple block') + [-0.3, 0]
put_first_on_second('purple bowl', target_pos)
```

4) LMPs can be composed.: Full prompt:

```
import numpy as np
from utils import get_pos, put_first_on_second, parse_obj
objs = ['yellow block', 'yellow bowl', 'gray block', 'gray bowl']
# move the sun colored block toward the left.
block_name = parse_obj('sun colored block')
target_pos = get_pos(block_name) + [-0.3, 0]
put_first_on_second(block_name, target_pos)
objs = ['white bowl', 'white bowl', 'yellow block', 'yellow bowl']
# place the block closest to the blue bowl on the other bowl.
block_name = parse_obj('the block closest to the blue bowl')
bowl_name = parse_obj('a bowl other than the blue bowl')
put_first_on_second(block_name, bowl_name)
```

5) *parse_obj* prompt.: Full prompt:

```
import numpy as np
from utils import get_pos
objs = ['brown bowl', 'green block', 'brown block', 'green bowl']
# the blocks.
ret_val = ['brown block', 'green block']
# the sky colored block.
ret_val = 'blue block'
objs = ['orange block', 'cyan block', 'purple bowl', 'gray bowl']
# the right most block.
block_names = ['orange block', 'cyan block']
block_positions = np.array([
    get_pos(block_name) for block_name in block_names])
right_block_name = block_names[np.argmax(block_positions[:, 0])]
ret_val = right_block_name
```

C. Reasoning with Code vs. Natural Language

To investigate how robot-relevant reasoning through LLMs can be performed with LMPs rather than with natural language, we created a benchmark that consists of two sets of tasks: (i) selecting objects in a scene from spatial-geometric descriptions, and (ii) selecting position coordinates from spatial-geometric descriptions. Object selection has 28 questions with commands such as “find the name of the block closest to the blue bowl,” where a list of block and bowl positions are provided as input context in the prompt.

Position selection has 23 questions with commands such as "interpolate 3 points on a line from the cyan bowl to the blue bowl." An LLM-generated answer for position selection is considered correct if all coordinates are within 1cm of the ground truth. We evaluate LMPs against two variants of reasoning with natural language: (i) **Vanilla**, given a description of the setting (e.g., list of object positions) and the question, directly outputs the answer (e.g., "Q: What is the top-most block?" → "A: red block"), and (ii) Chain of Thought (CoT) [47], which performs step-by-step reasoning given examples of intermediate steps in the prompt (e.g., encouraging the LLM to list out y-coordinates of all blocks in the scene before identifying the top-most block).

TABLE IV: Using code for spatial-geometric reasoning yields higher success rate (mean %) than using vanilla natural language or chain-of-thought prompting.

Tasks	Natural Language		Code
	Vanilla	CoT [47]	LMP (ours)
Object Selection	39	68	96
Position Selection	30	48	100
Total	35	58	98

Results in Table IV show that LMPs achieve accuracies in the high 90s, outperforming CoT, which outperforms Vanilla. CoT enables LLMs to reason about relations and orders (e.g. which coordinate is to the right of another coordinate), but failures occur for precise and multi-step numerical computations. By contrast, code from LMPs can use Python to perform such computations, and they often leverage external libraries to perform more complex operations (e.g., NumPy for vector addition). CoT and LMPs are not mutually exclusive – it is possible to prompt "step-by-step" code-generation to solve more complex tasks via CoT, but this is a direction not explored in this work.

D. CodeGen HumanEval Additional Results

Here we provide additional results to our HumanEval experiments. In total, three variants of the bigger Codex model (codedavinci-002) are tested. Our approach is Hier. CodeGen + Hier Prompts, where the prompt encourages the LLM to call yet-to-be-defined functions by including such examples. For comparisons, we evaluate against Flat CodeGen + No Prompt, essentially just using the LLM directly, and Flat CodeGen + Flat Prompt, for fair comparison with flat code-generation, since our hierarchical approach has a prompt. The prompts only contain only 2 Examples: Prompt for Flat CodeGen:

```
prompt_f_gen_flat = ""
def get_total(xs: List[float]) -> float:
    """Find the sum of a list of numbers called xs.
    """
    return sum(xs)
# end of function

def get_abs_diff_between_means(xs0: List[float],
                               xs1: List[float]) -> float:
    """Get the absolute difference between the means of two
    lists of numbers.
    m0 = sum(xs0) / len(xs0)
    m1 = sum(xs1) / len(xs1)
    return abs(m0 - m1) # end of function
```

Prompt for Hierarchical CodeGen:

```
def get_total(xs: List[float]) -> float:
    """Find the sum of a list of numbers called xs.
    """
    return sum(xs)
# end of function

def get_abs_diff_between_means(xs0: List[float],
                               xs1: List[float]) -> float:
    """Get the absolute difference between the means of two lists of
    numbers.
    """
    m0 = get_mean(xs0)
    m1 = get_mean(xs1)
    return abs(m0 - m1)
# end of function
```

Note the only difference in the hierarchical prompt is using a yet-to-be-defined function `get_mean` instead of calculating the mean directly. This "allows" the LLM to generate code that also call yet-to-be-defined functions. We report pass rates for when using the most likely outputs ("greedy", which is done by setting temperature to 0), as well as pass rates for at least one solution from sampling various numbers of solutions (1, 10, and 100) with temperature set to 0.8, similar to those used in prior works [1], [11], [58].

TABLE V: Hierarchical code generation also performs better (in % pass rates) on generic coding problems from the standard HumanEval benchmark [1]. For columns, Greedy means decoding LLM with temperature=0, while P@N means evaluating correctness across N samples decoded from LLM with temperature=0.8.

	Greedy	P@1	P@10	P@100
code-davinci-001 [11]	-	36.0	-	81.7
PaLM Coder [11]	-	36.0	-	88.4
Flat CodeGen + No Prompt	45.7	34.9	75.1	90.9
Flat CodeGen + Flat Prompts	50.6	36.6	77.6	93.3
Hier. CodeGen + Hier Prompts	53.0	39.8	80.6	95.7

See results in Table II. In all instances hierarchical code generation outperforms flat code generation, and the numbers achieved are higher than those reported in recent works [1], [11], [58] Note that we use code-davinci-002, while previous works use code-davinci-001, but the relative improvements with hierarchical are consistent across the board. Out of the 164 questions in HumanEval, 6.5% led to hierarchical code generation, but of which both Flat CodeGen variants got 44% success, while Hier CodeGen code got 56%. While success rate when sampling 100 responses is above 90% across the board, we note that sampling multiple solutions is not practical for LMPs, which need to perform tasks in a zero-shot manner without engineering prior unit tests. As such, for LMPs we always set temperature to 0 and use the most likely output.

E. Robot Code-Generation Benchmark

1) Example Questions: Here are four types of benchmark questions and their examples: • Vector operations with Numpy: `pts = interpolate_pts_np(start, end, n)` • Simple controls: `u = pd_control(x_curr, x_goal, x_dot, Kp, Kv)` • Manipulating shapes with shapely: `circle = make_circle(radius, center)` • Using first-party libraries:

```
ret_val = obj_shape_does_not_contain_others(obj_name,
other_obj_names)
```

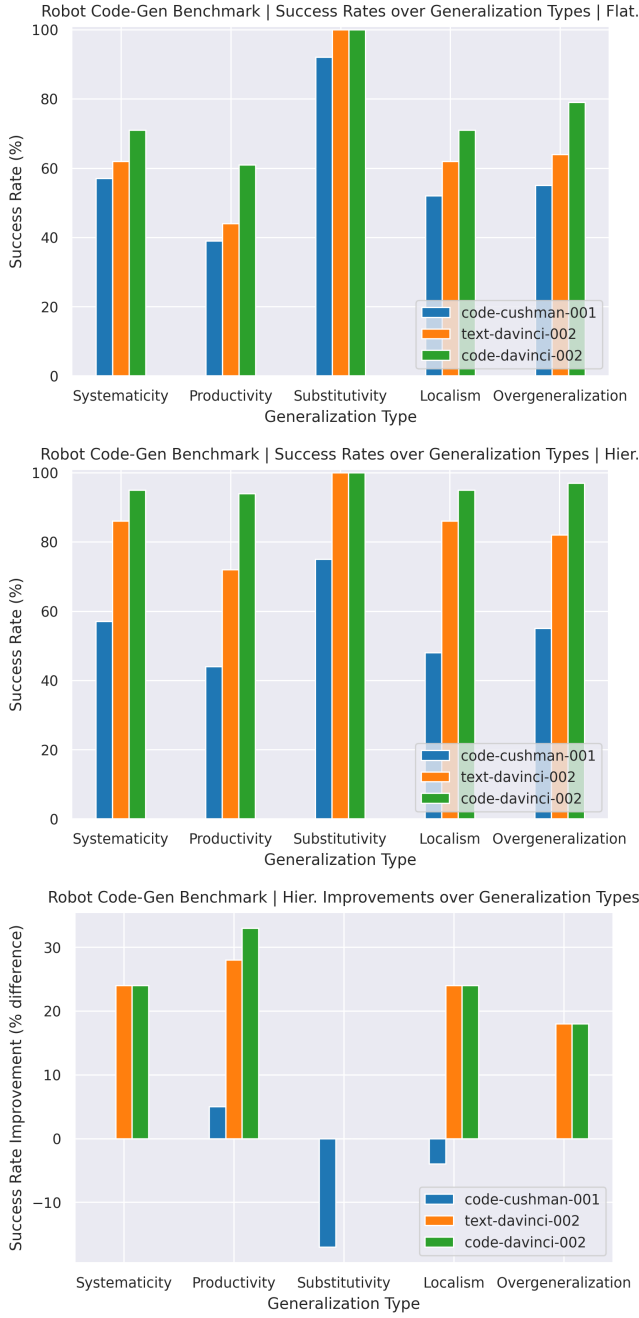


Fig. 4: Robot Code-Generation Benchmark Performance across Generalization Types for Flat (top) and Hierarchical (middle) Code- Generation, as well as the performance improvements made by Hierarchical Code-Generation (bottom).

For the last type, we provide imports of first-party functions, like ones that get object geometric information by name, as Hints in the prompt. 2) Generalization Analysis: We analyze how well codegeneration performs across the five types of generalizations described in [23], where generalization is evaluated by comparing the examples given in the prompt with the new instructions given in the benchmark. We give a description of the five types of generalization applied to our benchmark. Specifically, we say

that solving a problem in the benchmark demonstrates a particular type of generalization if the problem’s instruction or solution satisfy the following conditions:

- Systematicity: recompose parts of Examples’ instructions or code snippets.
- Productivity: have longer code or contains more levels (e.g., hierarchical function calls) than Examples.
- Substitutivity: use synonyms or replace words of similar categories from Examples.
- Localism: reuse seen parts or concepts for different purposes.

- Overgeneralization: use new API calls or programming language features not seen in Examples.

In Figure 4 we report results across three models. The top plot shows using flat prompts and flat code generation, while the bottom uses hierarchical prompts and hierarchical code generation. The relative rankings of the three models are consistent across all generalization types, with code-davinci-002 performing the best. The smallest model, code-cushman-001, performs especially poorly in Productivity and Localism, but it performs much better on Substitutivity. Indeed, the high Substitutivity performance across all models may indicate that language models are particularly robust to replacing words that have similar meanings or are from similar categories, while generalization to more complex commands (longer answers via Productivity or maintaining local structures via Localism) are more challenging. Hierarchical code generation also brings significant performance improvements for both davinci models, especially on Productivity, allowing these models to write longer and more multi-layered functions. However, these improvements are not apparent for cushman, which actually has lower Substitutivity and Localism success rates, suggesting reaching a certain code-generation capability level is needed before Hierarchical Code-Generation can bring further improvements.

F. CaP: Reactive Controllers for Toy Tasks

LMPs can express reactive controllers, and we demonstrate this on: (i) OpenAI Gym’s CartPole balancing and (ii) end-effector position impedance control. Using the same prompt structure, both controllers can be generated zero-shot, reflecting the knowledge already stored in the language model, which can be invoked when the objective is sufficiently specified in the function signature:

```
# define function: direction = keep_pole_upright_with_pd_control(x,
x_dot, theta, theta_dot).
# info: direction is 1 if going right, 0 if going left.
def keep_pole_upright_with_pd_control(x, x_dot, theta, theta_dot):
    # define constants.
    kp = 1
    kd = 1
    # define direction.
    direction = 1
    # define error.
    error = theta
    # define error_dot.
    error_dot = theta_dot
    # define control.
    control = kp * error + kd * error_dot
    # define direction.
    if control < 0:
        direction = 0
    return direction
```

The generated LMP expresses a controller that takes as input the position of the cart, angle of the pole, and their velocities, then it balances the pole by actuating the cart below with discrete left

or right actions. This controller is only a simple one that requires initialization to be near upright, but it can stabilize an unstable fixed point.

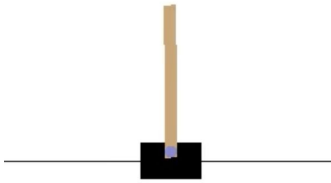


Fig. 5: LMPs can balance a cartpole

LMPs can likewise be prompted to express impedance control:

```
# define function: tau = ee_impedance_control(x_curr, x_goal,
      x_dot, K_x_mat, D_x_mat, J).
def ee_impedance_control(x_curr, x_goal, x_dot, K_x_mat,
      D_x_mat, J):
    x_err = x_goal - x_curr
    x_dot_err = -x_dot
    tau = np.matmul(J.T,
        np.matmul(K_x_mat, x_err) + np.matmul(D_x_mat, x_dot_err))
    return tau
```

to move a robot arm end-effector towards a goal position with joint torques. The controller is functional in that it can control a UR5e robot in PyBullet, but simplified in that it does not compensate for Coriolis or gravity forces. Note the need to include extra information about the expected direction output as well as the hint to use PD control in the function signature. Without these hints, the resultant function may still look reasonable (e.g. it may output continuous values for control instead of discrete), but it will not work for this specific environment API. For the names of the input gains, `_mat` was needed for the LMP to treat them as matrices instead of scalars, and `_x` was needed to indicate these gains were for the end-effector, not the joints. We demonstrate the use of this controller by commanding the end-effector 3D positions of a UR5e robot in PyBullet. The default PD gains of 1 also work in this domain without additional tuning as the CartPole environment is relatively simple. More complex continuous control tasks may require actually tuning the gains based on execution feedback, something our method does not support at the moment. Both examples show it is possible to generate simple reactive controllers, but more work is needed to express more complex ones.

G. Visual Language Models

For real-world experiments, we use off-the-shelf openvocabulary object detection models, ViLD [3] and MDETR [2] to perform object detection, localization, and segmentation. These are called visual language models because they take as input a natural language description (caption) of the image and try to find objects in that description. ViLD is used for the mobile robot domain, while MDETR is used for the tabletop manipulation and whiteboard drawing domains. Both models give an axis-aligned bounding box in the image along with per-pixel segmentation masks of the detected objects. To convert these detections to

3D coordinates for both perception and action (e.g., scripted picking primitives), we deproject the corresponding pixels from a depth camera, whose transform to the robot frame is registered a priori. The robustness of today’s vision language models could still be improved, and many real-world failures could be attributed to inaccurate detections. In addition, a degree of prompt engineering is also required for VLMs. For example, MDETR detects blocks more reliably with the word “square” than “block,” and applying our approach to a new domain will require some prompt engineering for the vision language model.

H. Whiteboard Drawing

In this domain, a UR5e robot is tasked to draw and erase various shapes described by natural language on a whiteboard. A dry-erase marker is rigidly attached to the robot end-effector. The whiteboard dimensions, location, and the location of the eraser are known. Additional objects may be added to the scene for the commands to refer to (e.g., draw a circle around the blue block). In our demos, we use Google Cloud’s speech-to-text and text-to-speech APIs to allow users interact with the system through voice commands and also hear the robot’s responses to commands. **Prompts.**

- `draw_ui`: the high-level UI for parsing user commands and calling other functions

https://code-as-policies.github.io/prompts/draw_ui.txt •

`parse_obj_name`: return names of objects from natural language descriptions

https://codeas-policies.github.io/prompts/parse_obj_name.txt

- `parse_shape_pts`: return sequence of 2D waypoints of shapes from natural language descriptions

https://codeas-policies.github.io/prompts/parse_shape_pts.txt

- `transform_shape_pts`: performs 2D transforms on a sequence of 2D points from natural language descriptions

https://codeas-policies.github.io/prompts/transform_shape_pts.txt •

`function_generation`: define functions from comments

https://code-as-policies.github.io/prompts/fgen_simple.txt

- `get_obj_names()` - gets list of available objects in the scene. these are prespecified.
- `get_obj_pos(name)` - get the 2D position of the center of an object by name.
- `draw(pts_2d)` - draws a shape by commanding the robot end-effector to follow a sequence of points on the whiteboard.

The robot first moves to a point above the first point in the trajectory, moves down to until contact with the whiteboard is detected, and proceeds to follow the rest of the trajectory.

- `erase(pts_2d)` - erases a shape by commanding the robot end-effector to first establish contact with a eraser (eraser position is hardcoded) before following the the rest of the trajectory.

Instructions. These **instructions** were given to the robot in series from an initial blank whiteboard. See full video and generated code on the website.

- 1) draw a 5cm hexagon around the middle
- 2) draw a line that bisects the hexagon
- 3) make them both bigger
- 4) erase the hexagon and the line
- 5) draw the sun as a circle at the top right
- 6) draw the ground as a line at the bottom
- 7) draw a pyramid as a triangle on the ground
- 8) draw a smaller pyramid a little bit to the left
- 9) draw circles around the blocks
- 10) draw a square around the sweeter fruit

I. Real-World Tabletop Manipulation

In this domain, a UR5e robot is tasked to manipulate objects on a tabletop according to natural language instructions. The robot is equipped with a suction gripper, and it can only perform pick and place actions parameterized by 2D top-down pick and place positions. The robot is also expected to answer questions about the scene (e.g., how many blocks are there?) by using the provided perception APIs. In our demos, we use Google Cloud's speech-to-text and text-to-speech APIs to allow users interact with the system through voice commands and also hear the robot's responses to commands and questions. Currently, the prompt only supports having a set of unique objects. This is not a limitation of CaP but rather of the perception system - we do not have a good way of persisting the identity of duplicate objects across VLM detections. A more sophisticated system of keeping track of the perceived world state can resolve this issue.

Prompts. • `tabletop_ui`: the high-level UI for parsing user commands and calling other functions

https://code-as-policies.github.io/prompts/tabletop_ui.txt • `parse_obj_name`: return names of objects from natural language descriptions

https://codeas-policies.github.io/prompts/parse_obj_name.txt • `parse_position`: return a 2D position from natural language descriptions

https://code-as-policies.github.io/prompts/parse_position.txt • `parse_question`: return a response (could be a number, a boolean, or a string) to a natural language question

https://codeas-policies.github.io/prompts/parse_question.txt • `function_generation`: define functions from comments

<https://code-as-policies.github.io/prompts/fgen.txt> APIs. • `get_obj_names()` - gets list of available objects in the scene. these are prespecified. • `get_obj_pos(name)` - gets the 2D position of the center of an object by name. • `is_obj_visible(name)` - checks if an object is visible by name. • `get_bbox(name)` - gets the 2D axis-aligned bounding box of an object by name. This is in robot base coordinates, not in pixels. • `get_segmask(name)` - gets the segmentation mask of an object detection by name. This is in pixels.

- `get_color_rgb(name)` - gets the average RGB color of an object detection crop by name.
- `get_corner_name(pos_2d)` - gets the name of the corner (e.g., top right corner) closest to the 2d point.
- `get_side_name(pos_2d)` - gets the name of the side (e.g., left side) closest to the 2d point.

`denormalize_xy(normalized_pos_2d)` converts a normalized 2D coordinate (each value between 0 and 1) to an actual 2D coordinate in robot frame. • `put_first_on_second(obj_name, target)` - picks the first object by name and places it on top of the target by name. The target could be another object name or a 2D position. Picking and placing are done by moving the suction gripper directly on top of the desired positions, moving down until contact is detected, then either engages or disengages the suction cup. • `say(message)` - uses the robot speaker to voice out a message. We demonstrate CaP on three domains in the tabletop manipulation setting. Instructions of each domain are listed below and were performed in a sequence. See full videos and generated code on the website. Instructions for 4 blocks domain.

- 1) Put the blocks in a horizontal line near the top
 - 2) Move the sky-colored block in between the red block and the second block from the left
 - 3) Why did you move the green block?
 - 4) Which block did you move?
 - 5) Arrange the blocks in a square around the middle
 - 6) Make the square bigger
 - 7) Undo that
 - 8) rotate the square by 45 degrees
 - 9) Can you throw blocks?
 - 10) Move the red block 5cm to the bottom
 - 11) Do the same with the other blocks
 - 12) Put the blocks on different corners clockwise starting at the top right corner
- Instructions for 3 blocks and 3 bowls domain.
- 1) Put the red block to the left of the rightmost bowl
 - 2) Now move it to the side farthest away from it
 - 3) How many bowls are to the left of the red block?
 - 4) place the blocks in bowls with non matching colors
 - 5) put the blocks in a vertical line 20 cm long and 10 cm below the blue bowl
 - 6) imagine that the bowls represent a volcano, a forest, and an ocean
 - 7) also imagine that the blocks are parts of a building
 - 8) now build a tower in the forest
 - 9) show me what happens when a volcano erupts over the ocean
- Instructions for fruits, bottles, and plates domain.
- 1) How many fruits are there?
 - 2) Tell me their names
 - 3) Are there any fruits on the green plate?
 - 4) Move all fruits to the green plate and bottles to the blue plate
 - 5) Move the smallest fruit back to the yellow plate
 - 6) Wait until you see an egg and put it on the green plate
 - 7) Put the darkest object in the plate that has the apple

J. Mobile Robot

The mobile manipulation experiment is set up with robots from Everyday Robots navigating and interacting with objects in a real world office kitchen. The robot has a mobile base and a 7DoF arm. For implementing the perception APIs, we mainly use the RGBD camera sensor on the robot. The robot is shown in Fig. 6.

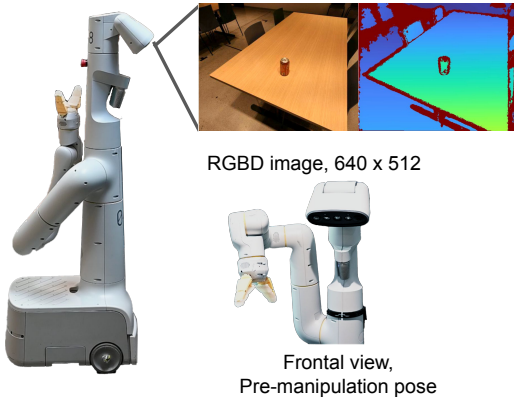


Fig. 6: Experiment Setup for mobile manipulation with a Everyday Robots robot.

Prompts.

- `mobile_ui`: the high-level UI for parsing user commands and calling other functions

https://code-as-policies.github.io/prompts/mobile_ui.txt • `parse_obj_name`: return names of objects from natural language descriptions

https://codeas-policies.github.io/prompts/mobile_parse_obj_name.txt • `parse_position`: return a 2D position from natural language descriptions

https://codeas-policies.github.io/prompts/mobile_parse_pos.txt • `transform_traj`: performs 2D transforms on a sequence of 2D points from natural language descriptions

https://codeas-policies.github.io/prompts/mobile_transform_traj.txt • `function_generation`: define functions from comments

https://code-as-policies.github.io/prompts/fgen_simple.txt APIs. • `get_obj_names()` - gets list of available objects in the scene. these are prespecified. • `get_obj_pos(name)` - get the 2D position of the center of an object by name. • `is_obj_visible(name)` - returns whether or not the robot sees an object by name. • `get_visible_obj_names()` - returns a list of currently visible object names. • `get_loc_names()` - returns a list of all predefined location names the robot can navigate to.

• `get_obj_pos(name)` - gets the 3D location of an object by name. This object must be currently visible. • `get_loc_pos(name)` - gets the 2D location and 1D angle of a predefined location. • `get_robot_pos_and_angle` - gets the current 3D robot position and 1D angle (heading).

• `goto_pos(pos_3d)` - navigates to a 3D position by running the robot's internal motion planner. • `goto_loc(name)` - navigates to a location by name by running the robot's internal motion planner. • `pick_obj(name)` - picks up an object by its name. The object must be currently visible. This is implemented as a scripted picking primitive using ViLD object detections. • `place_at_pos(pos_3d)` - places the currently held object at a position. • `place_at_obj(name)` - places the currently held object on top of another object by name. • `say(message)` - uses the robot's speaker to voice out a message. Below we list commands that were performed on the mobile robot platform. The first are navigation-related tasks, while the second are manipulation related. For the latter manipulation commands, note the ability of CaP to form "short-term memory" by explicitly record variables (in this case, the robot's past positions) in the Python execution scope and referring back them later. See videos and generated code on the website. Mobile Navigation Instructions. 1) Moving in a 3m by 2m rectangle around the office chair 2) Do that again but rotated 45 degrees clockwise 3) Go in a 1.5m square around the barstool as many times as needed, check each step if there is a banana, only stop moving when you see the banana 4) Follow the convex hull containing the chairs 5) Move back and forth between the table and the countertop 3 times Mobile Manipulation Instructions. 1) How many snacks are on the table? 2) Take the water bottle from the desk and put it in the middle of the fruits on the table 3) This is the compost bin 4) This is the recycle bin 5) This is the landfill bin 6) The coke can and the apple are on the table 7) Put way the coke can and the apple on their corresponding bins

K. Simulation Tabletop Manipulation Evaluations

Similar to the real-world tabletop domain, we construct a simulated tabletop environment, in which a UR5e robot equipped with a Robotiq 2F85 jaw gripper is given natural language instructions to complete rearrangement tasks. The objects include 10 different colored blocks and 10 different colored bowls. The proposed CaP is given APIs for accessing a list of present objects and their locations, via a scripted object detector, as well as a pick-and-place motion primitive that are parameterized by either coordinates or object names.

Prompts. • `tabletop_ui`: the high-level UI for parsing user commands and calling other functions

https://codeas-policies.github.io/prompts/sim_tabletop_ui.txt

- `parse_obj_name`: return names of objects from natural language descriptions

https://codeas-policies.github.io/prompts/sim_parse_obj_name.txt

- `parse_position`: return a 2D position from natural language descriptions

https://codeas-policies.github.io/prompts/sim_parse_position.txt

- `function_generation`: define functions from comments

<https://code-as-policies.github.io/prompts/fgen.txt> APIs.

- `get_obj_names()` - gets list of available objects in the scene. these are prespecified.
- `get_obj_pos(name)` - gets the 2D position of the center of an object by name.

- `denormalize_xy(normalized_pos_2d)` converts a normalized 2D coordinate (each value between 0 and 1) to an actual 2D coordinate in robot frame.
- `put_first_on_second(obj_name, target)` - picks the first object by name and places it on top of the target by name. The target could be another object name or a 2D position. Picking and placing are done by moving the suction gripper directly on top of the desired positions, moving down until contact is detected, then either engages or disengages the suction cup. We evaluate CaP and the baselines on the following tasks, where each task refers to a unique instruction template (e.g., “Pick up the `<block>` and place it in the corner `<distance>` to the `<bowl>`”) that are parameterized by certain attributes (e.g., `<block>`). We split the tasks into the instructions and the attributes to “seen” and “unseen” categories, where the “seen” instructions or attributes are permitted to appear in the prompt or used for training (in the case of supervised baselines). Full list can be found below. Note that we further group the instructions into “Long-Horizon” and “Spatial-Geometric” task families. The “Long-Horizon” instructions are 1-5 in Seen Instructions and 1-3 in Unseen Instructions. The “Spatial-Geometric” instructions are 5-8 in Seen Instructions and 4-6 in Unseen Instructions. Seen Instructions. 1) Pick up the `<block1>` and place it on the (`<block2>` or `<bowl>`) 2) Stack all the blocks 3) Put all the blocks on the `<corner/side>` 4) Put the blocks in the `<bowl>` 5) Put all the blocks in the bowls with matching colors 6) Pick up the block to the `<direction>` of the `<bowl>` and place it on the `<corner/side>` 7) Pick up the block `<distance>` to the `<bowl>` and place it on the `<corner/side>` 8) Pick up the `<nth>` block from the `<direction>` and place it on the `<corner/side>` Unseen Instructions. 1) Put all the blocks in different corners 2) Put the blocks in the bowls with mismatched colors 3) Stack all the blocks on the `<corner/side>` 4) Pick up the `<block1>` and place it `<magnitude>` to the `<direction>` of the `<bowl>`

5) Pick up the `<block1>` and place it in the corner `<distance>` to the `<bowl>` 6) Put all the blocks in a `<line>` line Seen Attributes.

1) `<block>`: blue block, red block, green block, orange block, yellow block 2) `<bowl>`: blue bowl, red bowl, green bowl, orange bowl, yellow bowl 3) `<corner/side>`: left side, top left corner, top side, top right corner 4) `<direction>`: top, left 5) `<distance>`: closest 6) `<magnitude>`: a little 7) `<nth>`: first, second 8) `<line>`: horizontal, vertical Unseen Attributes. 1) `<block>`: pink block, cyan block, brown block, gray block, purple block 2) `<bowl>`: pink bowl, cyan bowl, brown bowl, gray bowl, purple bowl 3) `<corner/side>`: bottom right corner, bottom side, bottom left corner 4) `<direction>`: bottom, right 5) `<distance>`: farthest 6) `<magnitude>`: a lot 7) `<nth>`: third, fourth 8) `<line>`: diagonal In Table VI we provide detailed simulation results that report task success rats for fine-grained task categories. Attributes refer to `<>` fields, the values of which can be seen by the method (e.g., training set for CLIPort, prompt for language-based methods). Instructions refer to the templated instruction type given in each row, which can also be seen or unseen. A total of 50 trials are performed per task, each with sampled attributes and initial scene configurations (block and bowl types, numbers, and positions). Note that CLIPort by itself (no oracle) is just a feedback policy and it does not know when to stop — in this case we run 10 actions from the CLIPort policy and evaluate success at the end. To improve CLIPort performance, we use a variant that uses oracle information from the simulation to stop the policy when success is detected (oracle termination).

L. Additional LLM Capabilities

Using a large pretrained LLM also means we can leverage its capabilities beyond code-writing. For example, Code as Policies can parse commands from non-English languages as well as emojis. See Figure 7.

M. Cross Embodiment Example

CaP exhibit a degree of cross-embodiment support [59], [60] by performing the same task differently depending on the action APIs. In the example below, we give Hints of the action APIs, and the resultant plan changes depending on the whether or not the robot is omnidirectional or unidirectional. We note that this ability is brittle with existing LLMs and cannot reliably adapt to APIs that are very different. More robustness may require larger ones trained on domain-specific code.

TABLE VI: Detailed simulation tabletop manipulation success rate (%) across different task scenarios.

	CLIPort (oracle termination)	CLIPort (no oracle)	NL Planner	CaP (ours)
Seen Attributes, Seen Instructions				
Pick up the <object1> and place it on the (<object2> or <recepticle-bowl>)	88	44	98	100
Stack all the blocks	98	4	94	94
Put all the blocks on the <corner/side>	96	8	46	92
Put the blocks in the <recepticle-bowl>	100	22	94	100
Put all the blocks in the bowls with matching colors	12	14	100	100
Pick up the block to the <direction> of the <recepticle-bowl> and place it on the <corner/side>	100	80	N/A	72
Pick up the block <distance> to the <recepticle-bowl> and place it on the <corner/side>	92	54	N/A	98
Pick up the <nth> block from the <direction> and place it on the <corner/side>	100	38	N/A	98
Total	85.8	33.0	86.4	94.3
Long-Horizon Total	78.8	18.4	86.4	97.2
Spatial-Geometric Total	97.3	57.3	N/A	89.3
Unseen Attributes, Seen Instructions				
Pick up the <object1> and place it on the (<object2> or <recepticle-bowl>)	12	10	98	100
Stack all the blocks	96	8	96	100
Put all the blocks on the <corner/side>	0	0	58	100
Put the blocks in the <recepticle-bowl>	46	0	88	96
Put all the blocks in the bowls with matching colors	30	26	100	92
Pick up the block to the <direction> of the <recepticle-bowl> and place it on the <corner/side>	0	0	N/A	60
Pick up the block <distance> to the <recepticle-bowl> and place it on the <corner/side>	0	0	N/A	100
Pick up the <nth> block from the <direction> and place it on the <corner/side>	0	0	N/A	60
Total	23.0	5.5	88.0	88.5
Long-Horizon Total	36.8	8.8	88.0	97.6
Spatial-Geometric total	0.0	0.0	N/A	73.3
Unseen Attributes, Unseen Instructions				
Put all the blocks in different corners	0	0	60	98
Put the blocks in the bowls with mismatched colors	0	0	92	60
Stack all the blocks on the <corner/side>	0	0	40	82
Pick up the <object1> and place it <magnitude> to the <direction> of the <recepticle-bowl>	0	0	N/A	38
Pick up the <object1> and place it in the corner <distance> to the <recepticle-bowl>	4	0	N/A	58
Put all the blocks in a <line>	0	0	N/A	90
Total	0.7	0.0	64.0	71.0
Long-Horizon Total	0.0	0.0	64.0	80.0
Spatial-Geometric Total	1.3	0.0	N/A	62.0

```

import numpy as np
from utils import get_pos, put_first_on_second
objects = ['cyan block', 'cyan bowl', 'pink bowl']
# put the cyan block in cyan bowl.
put_first_on_second('cyan block', 'cyan bowl')
objects = ['gray block', 'silver block', 'gray bowl']
# place the top most block on the gray bowl.
names = ['gray block', 'silver block']
positions = np.array([get_pos(name) for name in names])
name = names[np.argmax(positions[:,1])]
put_first_on_second(name, 'gray bowl')
objects = ['purple block', 'purple bowl']
# put the purple bowl to the left of the purple block.
target_pos = get_pos('purple block') + [-0.3, 0]
put_first_on_second('purple bowl', target_pos)
objects = ['red block', 'red bowl', 'blue block', 'blue bowl']
# 把蓝色的木块放在红色的碗里。
put_first_on_second('blue block', 'red bowl')
objects = ['green block', 'cyan block']
# move the 🟩 10cm to the 🏠.
target_pos = get_pos('green block') + [-0.1, 0]
put_first_on_second('green block', target_pos)

```

```

# omnidirectional robot.
# available actions: move_up(dist), move_right(dist), move_back(dist).
# make a U shape 5 meters wide and 10 meters long.
robot.move_back(dist=10)
robot.move_right(dist=5)
robot.move_up(dist=10)
# unidirectional robot.
# available actions: turn_left(deg), move_forward(dist).
# make a U shape 3 meters wide and 8 meters long.
robot.move_forward(dist=8)
robot.turn_left(deg=90)
robot.move_forward(dist=3)
robot.turn_left(deg=90)
robot.move_forward(dist=8)

```

Fig. 7: LMPs inherit benefits of LLMs, such as parsing commands from non-English languages and emojis.